Lab 08: Textures

```
running build\Release\info.exe
  device         Intel(R) UHD Graphics
  driver         Intel Corporation Intel driver
  api            1.2.187
  timestamps     52.0833 ns per tick
  anisotropy     samplerAnisotropy enabled, maxSamplerAnisotropy 16
  blend          independentBlend yes (two attachments, two different blends)
  srgb           R8G8B8A8_SRGB sampled with linear filtering: yes
  float targets  blending into R16G16B16A16_SFLOAT yes, R16_SFLOAT yes, R8_UNORM yes
  queryLod       textureQueryLod works: lod 2.00 (raw 2.00) for 4 texels a pixel, expected 2.00
  glslc          C:/VulkanSDK/1.4.357.0/Bin/glslc.exe
  version        shaderc v2026.3 v2026.3
  shaders        D:/Engineering/Year_III/Computer_Graphics/assignments/lab8/lab-08-computer-graphic/shaders
  color.glsl           TASK 1 done
  normalmap.vert       TASK 2 done
  normalmap.frag       TASK 2 done
  miplevel.frag        TASK 3 done
  wboit_accum.frag     TASK 4 done
  wboit_composite.frag TASK 4 done
```

Part I:

```
running build\Release\srgb.exe
  shaders        fullscreen.vert + srgb_tiles.frag for the tiles,
                 srgb_wall.vert + srgb_wall.frag for the wall, both include color.glsl
  texture        256x256 generated, the same bytes twice: tex0 SRGB, tex1 UNORM
  tiles          A x 8: hardware decode   B x 272: your decode   C x 536: no decode
  decode         texel 128 -> linear 0.2159 (byte 55)   yours 55   hardware 55   PASS
  ramp           max |yours - hardware| over 256 levels  1 (texel 60)   PASS (<= 1)
  roundtrip      max |encode(decode(x)) - x|  0 with your decode, 0 with the hardware's   PASS
  no decode      texel 128 displays as 188 (should be 128), cpu 188   PASS
  range          too dark 3.8% (cpu 3.8%)   too bright 2.6% (cpu 2.6%)   PASS
                 luminance of the linear colour, the albedo half of tile B
  per channel    the same limits on each channel alone (cpu): too dark 5.1%, too bright 2.6%
  wall           8 x 5 m, 256x256 bricks generated, repeated 4 x 2.5, camera at (0.50, 0.72, 5.15)
                 one upload, 9 box mips, trilinear, repeat: tex0 its SRGB view, tex1 its UNORM view
  spotlights     A warm (1.00, 0.62, 0.32) x 1.00 above, B cool (0.32, 0.58, 1.00) x 1.20 below
                 aimed at (0.50, 0.55) and (0.50, -0.55) m from 6 m out, no ambient
                 cone 4.5 to 10.2 deg, falloff 1 / d^2, both on the split column
                 yours  left of the split: linear_to_srgb(srgb_to_linear(raw) * (A + B))
                 gamma  right of it, mode 0: raw * (linear_to_srgb(A) + linear_to_srgb(B)),
                        each light tuned alone in gamma space
                 cpu    yours in double from raw, the wall as stored (knob 1)
                 split at 800 for yours, at 0 for gamma, time 1; probes in column 390, on clay bricks
  spot A alone   (390, 223)  yours (172,  62,  34)  cpu (172,  62,  34)  PASS   gamma (172,  64,  38)
  overlap        (390, 296)  yours (186,  99,  82)  cpu (186, 100,  82)  PASS   gamma (255, 142, 110)
  spot B alone   (390, 374)  yours (102,  80,  76)  cpu (103,  81,  76)  PASS   gamma (104,  81,  75)
                 shapes (A, B) (1.00, 0.00) (0.98, 0.95) (0.00, 1.00), PASS within 2 levels
  gamma = yours? A alone yes (max 4), overlap no: gamma up to +69 and clipped, B alone yes (max 2)
                 yes = within 4 levels (sRGB is no pure power: enc(a b) is only nearly enc(a) enc(b))
  image          MATCH (0.00% differ)   (wall, split 400, mode 0)
  gpu time       0.289 ms median, 0.294 ms p95
```

Yanaphat Chatphokhinkun 6731313421

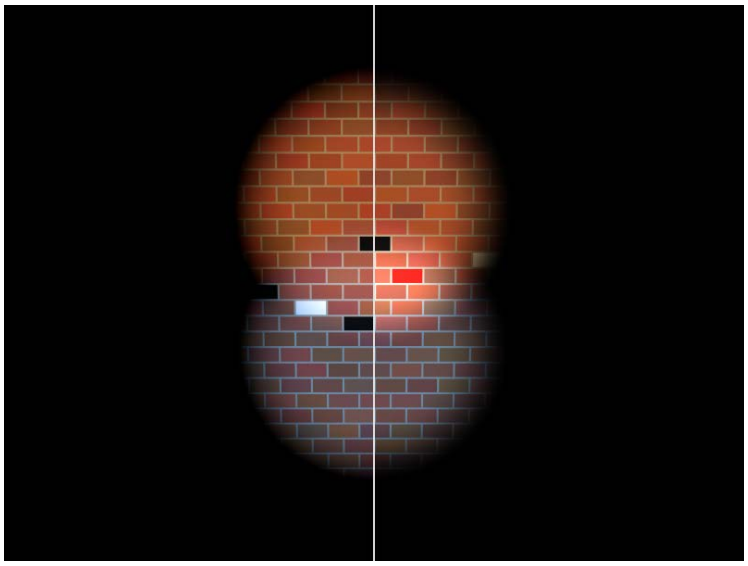
Write-up 1:

1. Your no-decode tile shows texel 128 as 188. How many times too much light does a midgrey wall reflect if the decode is skipped? Work it out from Equation 1. The chapter says “about 2×” and uses 0.5^2.2. Give the exact factor and the factor with x^2.2.

Ans The encoded value of texel 128 is c = 128/255 = 0.50196. Equation 1 gives L(c) = ((0.50196 + 0.055) / 1.055) ^ 2.4 = 0.21586 Skipping decoding treats 0.50196 as linear reflectance, so the wall reflects 0.50196 / 0.21586 = 2.3254 times the correct light. Using c ^ 2.2 on the same input gives 2.2866 times. Using the chapter’s rounded input of 0.5 gives 2.2974 times. Therefore, “about 2×” understates the error. The measured no-decode tile displayed byte 188, as expected.

2. The wall in mode 0. On some brick, each spot on its own displays the byte 150, on both sides of the split. What does the overlap of the two spots display on the left, and what on the right? Where a pool has faded to a quarter of its centre's light, what does each side display? Work both out from Equation 1, then compare your screenshot. Why can an artist working in a gamma pipeline make one light look right, but never two?

Ans



Byte 150 corresponds to linear light L(150/255) = 0.3049873. Adding two contributions in linear space and encoding gives 255 x E(2 x 0.3049873) ~ 205.

The gamma-space pipeline adds 150 + 150 = 300, which clips to 255. At one quarter of the centre's light, the correct pipeline gives: 255 x E(0.3049873/4) ~ 78. The supplied gamma-space shader multiplies its encoded spotlight contribution by the falloff, giving 150/4 = 37.5, approximately 38. Consequently, its edges become too dark and its overlap becomes too bright. The mode 0 shows this pattern. At the measured overlap pixel, the correct result was RGB(186,99,82), while gamma-space lighting produced RGB(255,142,110). An artist can match an individual light at one brightness, but nonlinear encoding does not preserve addition E(A+B) != E(A)+E(B).

3. A roughness map is loaded as SRGB by mistake. Is a roughness of 0.5 read as rougher or smoother, and by how much? Why is no image ever wrong this way for a metallic map that only holds 0 and 1?

Ans The sampler decodes 0.5 to ((0.5 + 0.055) / 1.055) ^2.4 = 0.21404. The material becomes smoother because its roughness parameter decreases by 0.28596, or 57.2%. A metallic map containing only 0 and 1 is unaffected because sRGB decoding preserves both endpoints. Intermediate gray values would change.

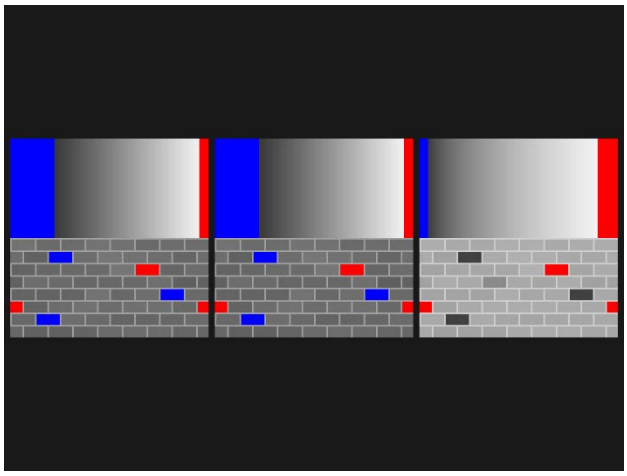
4. The chapter's Python check (section 9.6.2) tests each channel separately against [0.04, 0.9]. Yours tests the luminance of each pixel. Find a texel in the test texture that one check flags and the other passes. Which check is right for a saturated red brick, and why?

Ans A texel inside the saturated red brick, at texture coordinate (88,184), contains RGB(200, 30, 25). Its decoded linear RGB is approximately (0.57758, 0.01298, 0.00972) The per-channel check flags it as too dark because green and blue are below 0.04. However, its luminance is Y = 0.2126R+0.7152G+0.0722B = 0.13278 This is inside the permitted range. The luminance check therefore accepts it and displays RGB(102, 102, 102) in the debug view. Luminance is appropriate for this overall brightness check: low green and blue values are expected for a saturated red material. The measured too-dark percentage was 3.8% using luminance and 5.1% using individual channels. The additional 1.3 percentage points come from the red brick.

5. Generate an albedo texture with any image model (or download one), run Part I on it with --albedo yours.png, and look at mode 1. What fraction of it is too dark or

too bright, and what in the picture caused it? Is there light and shadow painted into it (section 9.6.1, “baked lighting”)?

Ans



The tested image was the 1024×1024 diffuse map from Poly Haven’s Brick Wall 001. Mode 1 reported 12.4% too dark and 0.0% too bright, matching the CPU. In the albedo screenshot, blue pixels mainly correspond to dark, soot-like patches on brick faces and some cracks and recessed edges. No sampled pixels exceeded the upper luminance limit. The source image has no obvious large directional cast shadow or bright specular highlight. Some crack and edge shading suggests possible residual baked occlusion, but dark patches alone cannot establish that lighting is baked in: they may represent dark material or staining. The brightness check identifies suspicious regions that need visual inspection.

Part II:

```
\make.bat normalmap
  probe         (382, 253) up-left     cpu (-0.43,  0.79,  0.45)  yours (-0.42,  0.79,  0.45)  dot 1.
0000   PASS
  probe         (313, 258) left        cpu (-0.89,  0.21,  0.41)  yours (-0.88,  0.22,  0.42)  dot 0.
9999   PASS
  probe         (314, 334) down-left   cpu (-0.50, -0.65,  0.58)  yours (-0.50, -0.65,  0.58)  dot 1.
0000   PASS
  probe         (389, 336) down-right  cpu ( 0.34, -0.64,  0.69)  yours ( 0.34, -0.64,  0.69)  dot 1.
0000   PASS
  convention    DX map + flip (knob 1) vs GL map    0.00% of the sphere differs   PASS (< 0.5%)
  no flip       DX map, knob 0 vs GL map           57.18% of the sphere differs   PASS (> 10%)
  mean |n|      mip 0  1.0000   mip 2  0.9979   mip 4  0.9645   mip 6  0.8902
                n = byte / 255 * 2 - 1 over every texel of the level (cpu)
  image         MATCH (0.29% differ)
  gpu time      0.253 ms median, 0.264 ms p95
  keys          0..3 debug modes, S saves a png, Esc quits
                left-drag orbits the camera, the scroll wheel moves it in and out
                C binds the OpenGL map or its DirectX twin (green inverted)
                K toggles the knob: 0 reads the map as OpenGL, 1 as DirectX (flip green)
                G binds the normal map through its SRGB view (the classic mistake) or UNORM
                L pauses and restarts the light going round the sphere
                [ and ] lower and raise the forced level of detail of mode 3 by 0.5
```

Write-up 2:

1. Bind the DirectX map without the flip and let the light circle the sphere (the chapter's 360° test). Describe exactly what goes wrong and when: at which light positions do the bumps look right, and at which do they look like dents? Explain this from Equation 2.

Ans Reading the DirectX map without flipping green reverses the tangent-space y component. Equation 2 should produce

Nworld = Normalize(Tn_x + Bn_y + N_gn_z)

The incorrect interpretation instead uses -Bn_y. Therefore, the slope along the bitangent is reversed, while the tangent and outward-normal components keep their signs. This swaps the bright and dark sides along that direction and makes raised bumps resemble dents. In the sampled orbit, the front-and-above light at 90° produced a clear reversal of the upper/lower shading. With light from the sides at 0° and 180°, the left/right response remained plausible, but the vertical response was still wrong. At 270°, the light was behind the sphere, and ambient lighting hid much of the error on the front. The error becomes small where B dot L is near zero and strongest where light has a large component along B. Because the tangent frame varies across the sphere, no single angle makes every bump correct. At the

default angle, the panel measured 57.18% different normals without the flip and 0.00% with it.

2. Bind the correct map as SRGB (key G). Which way do the normals tilt, and why does a flat area, (128, 128, 255), stop being flat? Compute the normal that this texel turns into.

Ans An SRGB view applies color decoding to values that represent direction data. The normalized bytes are approximately (0.50196, 0.50196, 1). Equation 1 decodes these to (0.21586, 0.21586, 1). Applying the normal-map conversion 2t-1 gives (-0.56828, -0.56828, 1). After normalization, the tangent-space direction becomes (-0.44296, -0.44296, 0.77947) It tilts toward -T and -B, approximately 38.8° away from the outward normal. In a UNORM view, the same bytes produce approximately (0.00392, 0.00392, 1), which is nearly flat. The SRGB mistake therefore biases supposedly flat regions away from the light, explaining the much darker screenshot.

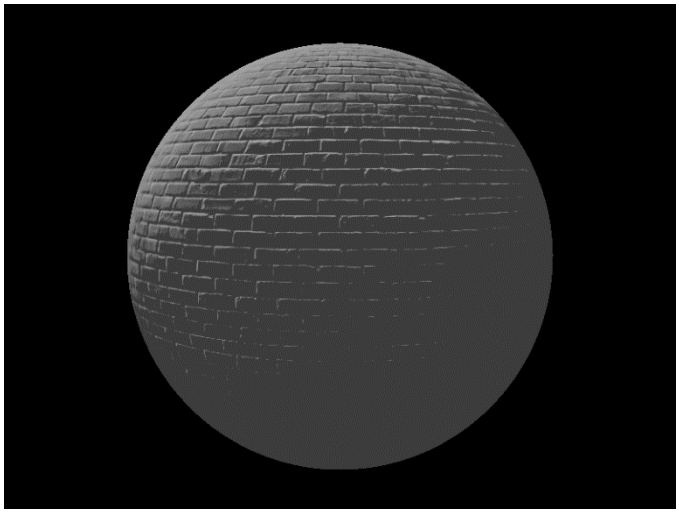
3. The panel prints the mean length of the decoded normal at mip 0, 2, 4 and 6. Why does it shrink, and why does the chapter say to renormalise normal-map mips? After you renormalise, what has still been lost about the surface?

Ans The measured mean lengths were 1.0000, 0.9979, 0.9645, and 0.8902 at mip levels 0, 2, 4, and 6. Mip generation averages neighboring normal vectors. Those vectors point in different directions, so their sideways components partly cancel and the resulting vector becomes shorter. Higher mip levels combine more surface variation, producing greater cancellation. Renormalization restores a unit direction for lighting. Otherwise, the shortened vector would incorrectly reduce the diffuse dot product. However, it cannot recover the original bumps, individual slopes, or variation among those slopes. Opposing slopes may average to a vector pointing straight outward; normalizing that vector produces a flat normal rather than restoring the bump. The lost variation also matters for effective roughness and specular highlights.

4. Run the 360° test on a normal map from an image model or from a texture site, with --normal yours.png. Which convention is it, and how did the test tell you?

Ans

Yanaphat Chatphokhinkun 6731313421



The tested texture was the Normal (GL) map from Poly Haven’s Brick Wall 001. It uses the OpenGL convention, so this lab should read it through UNORM with knob = 0, without flipping green. The orbit test compared both interpretations at eight light positions. Without the flip, the bricks appeared raised above the recessed mortar and their lit edges followed the light. Flipping green reversed the upper/lower edge response, giving horizontal mortar grooves an inverted-relief appearance, especially with front-and-above lighting. The left/right response remained similar, identifying the green-channel sign as the problem. This agrees with the source’s nor_gl label. The panel’s green-inverted twin matched the original with 0.00% difference after correction; leaving that twin uncorrected changed 88.68% of sphere pixels.

Part III:

```
computer-graphic> .\make.bat floor --mode 1
  row y=270      cpu lod  7.23 ratio 12.62   yours  7.23 12.62   hw  7.23   PASS   12.9 m away
  tolerance      lod within 0.05 levels, ratio within 3%;  hw and cpu at most 0.00 levels apart
  ratio > 4      yours on 37.7% of the floor pixels, cpu 37.8%   PASS
                 cpu: from row 357 up in the middle column (3.9 m away),
                 from row 391 up at the left edge (3.9 m away)
  sweep          GPU ms a frame, 16 texture() calls a pixel, 3 rounds
                 filter         median   each round              vs trilinear
                 nearest        0.809    0.809  0.809  0.810      0.96x
                 bilinear       1.200    1.202  1.199  1.200      1.42x
                 trilinear      0.844    0.843  0.844  0.847      1.00x
                 aniso4         3.353    3.353  3.346  3.363      3.97x
                 aniso16        5.708    5.702  5.708  5.716      6.76x
                 nearest and bilinear have no mips: they read level 0 at every distance
  image          DIFFERS (4.21% differ)   (mode 0, aniso16, 1 texture() call)
  gpu time       0.385 ms median, 0.403 ms p95
  keys           0..3 debug modes, S saves a png, Esc quits
                 left-drag orbits the camera, the scroll wheel moves it in and out
                 mode 0 textured, 1 mip level bands, 2 anisotropy heat map, 3 probe encoding
                 F cycles the filter: nearest, bilinear, trilinear, aniso4, aniso16
                 T switches between 1 and 16 texture() calls a pixel
  watching       D:/Engineering/Year_III/Computer_Graphics/assignments/lab8/lab-08-computer-graphic/shaders
```

Prediction

I predict the filters will rank from cheapest to most expensive as nearest → bilinear → trilinear → anisotropic ×4 → anisotropic ×16. I estimate anisotropic × 16 will cost approximately 1.5 times as much as trilinear. Filtering generally requires more texture samples as quality increases, but texture caching and the fact that many floor pixels do not need the maximum anisotropy should keep the overall increase well below 16 times.

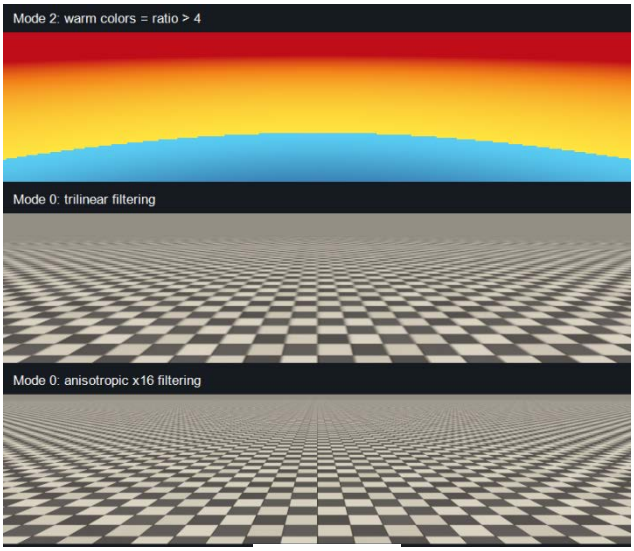


Figure 1

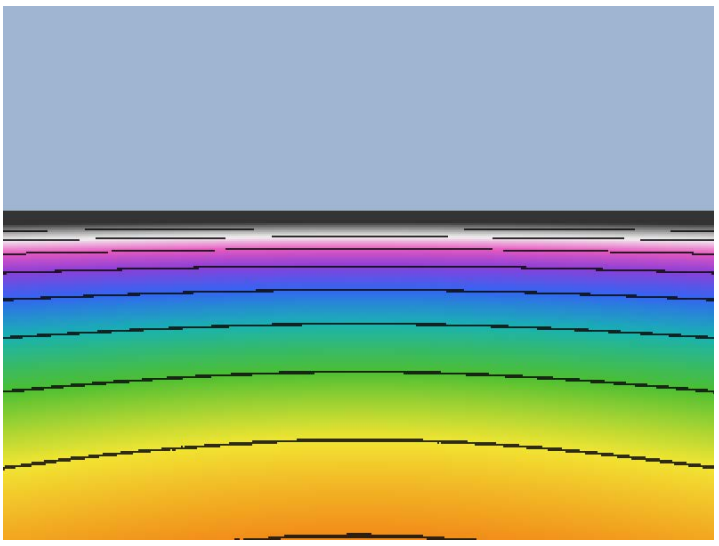


Figure 2

Write-up 3:

1. In your mode 2 picture, where is the anisotropy ratio above 4? Show that this is the same region where trilinear blurs the grout lines in mode 0. Use Equation 4 to explain why it sits there.

Ans The warm-colored region near the horizon has a ratio above 4: 37.7% of floor pixels, compared with the CPU's 37.8%. The boundary is around row 357 at the center and row 391 at the left edge. Equation 4 divides the longer texture-footprint dimension by the shorter. At this shallow viewing angle, one vertical screen pixel spans much more texture than one horizontal pixel, creating an elongated footprint. Trilinear chooses a mip from the longer dimension, over-blurring the shorter direction. Figure 1 shows the corresponding loss of grout detail; anisotropic ×16 preserves more detail.

2. Your timing table against the chapter: section 9.2.2 says anisotropic ×16 costs about 15% more than trilinear. What does it cost on your GPU, and how many texels does each filter actually read per pixel in the region where the ratio is 16?

Ans Anisotropic ×16 took 6.246 ms versus 0.918 ms for trilinear: 6.80× the time, or about 580% more, rather than the chapter's 15%. These are medians across three rounds of the supplied sweep, using 16 texture() calls per floor pixel.

Filter	Median (ms)	Relative cost	Texels/call*
Nearest	1.079	1.17×	1
Bilinear	1.428	1.55×	4
Trilinear	0.918	1.00×	Up to 8
Anisotropic ×4	3.844	4.19×	Up to 32
Anisotropic ×16	6.246	6.80×	Up to 128

3. The mip-level bands in mode 1 are not straight lines across the screen. Why do they bend towards the sides? Answer with Equation 3.

Ans Equation 3 gives $\lambda = \log_2(\max(|\partial t/\partial x|, |\partial t/\partial y|))$, where $t = uv \times texSize$. Away from the center, moving vertically on screen also moves sideways across the floor because of perspective. This increases the texture derivative's length and therefore the mip level. A given mip boundary occurs lower in the image at the sides, creating the curved bands in Figure 2.

4. Your prediction for the sweep against the measurement. Where were you wrong?

Ans I predicted nearest → bilinear → trilinear → ×4 → ×16, with ×16 costing 1.5 × trilinear. The measured order was trilinear → nearest → bilinear → ×4 → ×16. Trilinear was fastest in all three rounds; its smaller mip levels plausibly improve cache reuse compared with always reading level 0. I also underestimated ×16: 6.80× rather than 1.5×. Sample count alone did not predict this GPU's performance.

Part IV:

```
                mean |unsorted - sorted| 5.9 levels, max 114
weight          w over the transparent objects, 4.3 .. 30.0 m from the camera
                chapter, window depth 0.0023 .. 0.0223   w 3000.0 .. 3000.0
                view depth in metres                  w 58.2 .. 2937.3
                (a depth buffer with 0 near, 1 far: z 0.9777 .. 0.9977, w 2999.8 .. 2999.8)
fp16 accum      largest accum.a, and how many pixels have a channel at 65504
                (the largest fp16) or at Inf / NaN
                weight on      1000 particles     10000 particles
                view depth        6972 (0 px)        13704 (0 px)
                window depth      8896 (0 px)        46688 (0 px)
                Inf or NaN: none
timing          GPU ms of the transparent passes (median of 3 rounds); sort on the CPU
                particles   unsorted   sorted: cpu sort + gpu   wboit: both passes
                     1000      0.099         0.025 +  0.100          0.226
                    10000      0.850         0.874 +  0.807          1.695
image           MATCH (0.01% differ)
gpu time        0.367 ms median, 0.385 ms p95 (whole frame, wboit)
keys            0..3 debug modes, S saves a png, Esc quits
                left-drag orbits the camera, the scroll wheel moves it in and out
                mode 0 picture, 1 revealage (magenta: fp16 accum saturated),
```

Prediction:

From 1,000 to 10,000 particles, I predict sorted transparency will have the largest increase in total time (CPU sort + GPU rendering). It adds an O(n log n) CPU sort to the extra rendering work, while unsorted blending and WBOIT avoid sorting. WBOIT will still cost extra GPU time for two accumulation targets and a composite pass.

Figure 1:

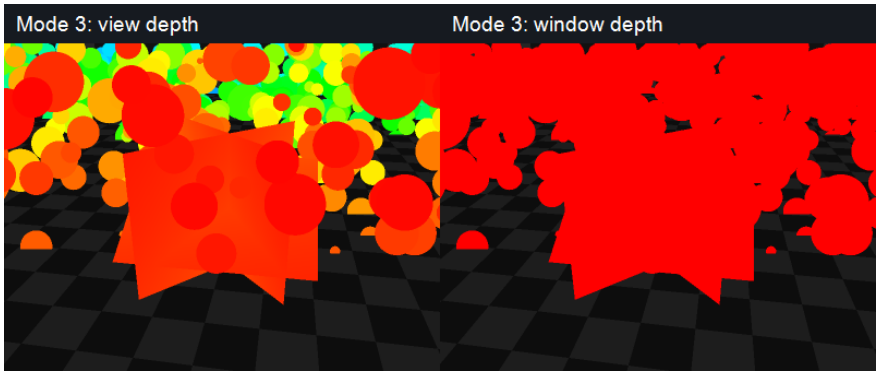


Figure 2:

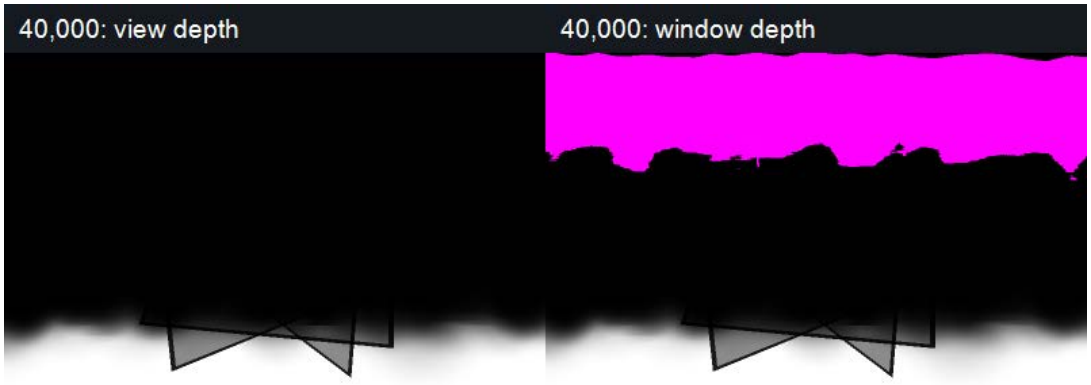
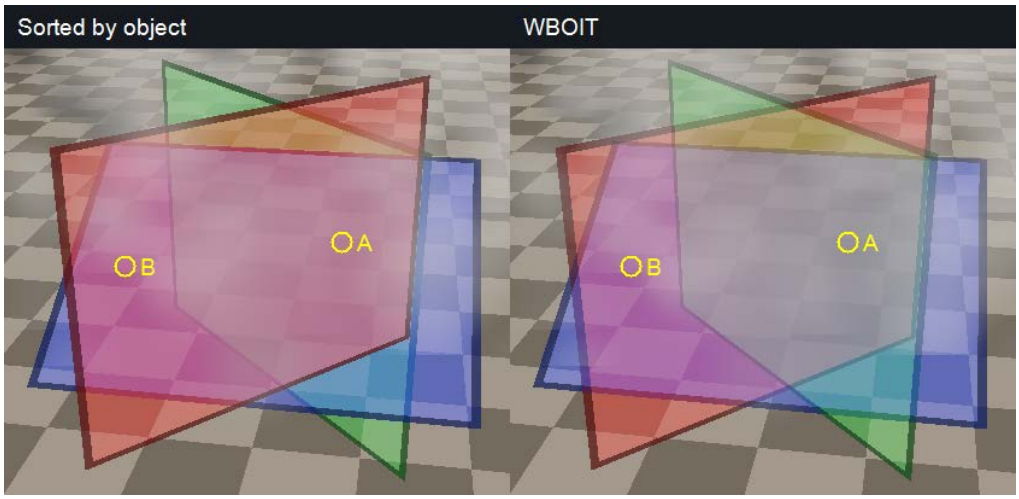


Figure 3:



Write-up 4:

1. The chapter's WBOIT shader (section 9.5.3) computes the weight from in.position.z, the depth in the depth buffer, which is between 0 and 1. Put that into Equation 7. What weight does every fragment get? Show it with the panel's weight line and a mode 3 screenshot of both weights (key W), then say what the chapter's version of WBOIT reduces to. In this scene, the two pictures differ by only a few

levels on average. Why so little here? Then run make oit ARGS="--particles 40000 --mode 1" with each weight (key W). Magenta marks pixels where fp16 accumulation saturated at 65504. Which weight saturates, and why?

Ans For z between 0 and 1, Equation 7 gives approximately 2999.81–3000: almost one constant weight. The panel shows 3000.0–3000.0 for window depth, versus 58.2–2937.3 for view depth. The constant cancels in Equation 6, leaving an alpha-weighted average color with revealage opacity. The images differ by only 3.31 RGB levels on average: the nearby panes have similar weights, and much of the distant smoke has similar neutral colors. At 40,000 particles, window-depth weights saturate 119,302 pixels at 65504; view-depth weights saturate none (maximum accum.a = 37504). Large constant contributions overflow fp16 as layers accumulate.

2. Two layers of $\alpha = 0.5$, red in front of blue (Section 3). Prove that weights in the ratio wfront : wback = 2 : 1 make WBOIT exact. For n layers, what should the ideal weight of a layer be? Why can no function of z alone produce it?

Ans With red and blue alpha = 0.5, total opacity is $1 - 0.5^2 = 0.75$. Weights 2:1 give color shares 2/3 and 1/3, so the result is 0.5 red + 0.25 blue, exactly "over"; the background contributes 0.25. For n front-to-back layers, ideal weights are proportional to $T_i = \prod_{j<i}(1 - \alpha_j)$. Since $\Sigma\alpha_i T_i = 1 - \prod(1 - \alpha_i)$, Equation 6 then reproduces exact compositing. These weights depend on the layers and opacities in front, not only depth z.

3. The three glass panes cut through each other. Why can no back-to-front order of the three objects be right? Point to the place in your sorted screenshot where it fails.

Ans The panes intersect, so their front-to-back order changes across pixels. At A, pixel (460, 280), green is in front of blue and red, but center-based sorting draws red last and makes the overlap too red. On the left, red really is in front. One order for whole objects cannot satisfy both regions (Figure 3).

4. Your timing table: unsorted, sorted (the sort on the CPU plus the GPU) and WBOIT, at 1000 and 10000 particles. Which grows fastest, and why? Was your prediction right?

Ans The table uses the official three-round medians. Sorted totals add CPU sorting to GPU time; the CPU/GPU split was 0.027/0.907 ms at 1,000 particles and 0.648/8.132 ms at 10,000.

Method	**1,000 (ms)**	**10,000 (ms)**	**Increase (ms)**
Unsorted	0.903	8.159	7.256

Method	1,000 (ms)	10,000 (ms)	Increase (ms)
Sorted: CPU + GPU	0.934	8.780	7.846
WBOIT	1.570	12.979	11.409

WBOIT has the largest absolute increase (11.409 ms), so my prediction was wrong. Overlapping fragments require repeated blending into two targets plus a composite pass; GPU work dominates this result. Sorted has the largest proportional growth (9.40× versus WBOIT's 8.27×), but I predicted the absolute increase.

5. Find a place in your WBOIT picture where it is visibly wrong compared to the sorted one. Say why, using Equation 6.

Ans At B, pixel (280, 300), red is in front of blue and sorting gives the correct pane order. WBOIT makes this area more purple: RGB changes from (185, 110, 154) to (174, 119, 168). Equation 6 averages colors with approximate depth weights instead of the exact front-layer transmittance, giving the blue layer too much influence.

AI Prompt Log Overview

Part I — Color and sRGB

Prompt: Explain the instruction and write-up part I.

Model: OpenAI Codex (GPT-6)

Idea: I needed help distinguishing encoded colors from linear-light values and interpreting the albedo and lighting checks.

Verification: I tried to read the instruction along with the AI's answer, and verify based on the relevance and context that it gives.

Part II — Normal mapping

Prompt: Implement normalmap.vert and normalmap.frag, investigate the image mismatch, prepare Write-up 2, and explain the custom-map orbit test.

Model: OpenAI Codex (GPT-6)

Idea: I needed help transforming tangent-space normals into world space, handling the green-channel convention, and explaining incorrect sRGB decoding and mipmap averaging.

Verification: All six normal probes passed. Flipping the DirectX map matched the OpenGL map with 0.00% difference; omitting the flip changed 57.18% of sphere pixels. The default Intel reference image matched at 0.29%, not exactly zero. Orbit captures compared both green-channel conventions on a downloaded OpenGL brick normal map.

Part III — Mip levels and anisotropy

Prompt: Implement miplevel.frag, investigate the 4.21% mismatch, repair the source, and prepare a shorter Word write-up.

Model: OpenAI Codex (GPT-6)

Idea: I needed help calculating texture footprints from derivatives and relating mip selection and anisotropy to image detail and GPU cost.

Verification: After rebuilding, all six CPU probes and the coverage check passed: 37.7% of floor pixels had ratio > 4 versus CPU 37.8%. The mode 0 image still differed by 4.21% on Intel; GPU comparisons supported a filtering-dependent cause. The recorded prediction was 1.5× trilinear for anisotropic ×16; measurement was 6.80×.

Part IV — Weighted blended transparency

Prompt: Implement wboit_accum.frag and wboit_composite.frag, record a timing prediction, and create write4-short.docx.

Model: OpenAI Codex (GPT-6)

Idea: I needed help accumulating weighted colors and revealage, deriving ideal layer weights, and comparing sorting errors, approximation errors, saturation and performance.

Verification: Single-layer error was 0 levels; the two-layer red share and revealage matched CPU values 0.844 and 0.280. Reversing order changed WBOIT by at most 1 level. The reference image matched at 0.01%. At 40,000 particles, window-depth weights saturated 119,302 pixels, while view-depth weights saturated none. The prediction that sorting would have the largest absolute time increase was wrong; WBOIT did.